```
const path = require('path');
const { fetchMovies, fetchSingleMovie } = require('./consumer');
const { PactV3, MatchersV3 } = require('@pact-foundation/pact');

const {
  eachLike,
  integer,
  string,
} = MatchersV3;

const provider = new PactV3({
  dir: path.resolve(process.cwd(), 'pacts'),
  consumer: 'WebConsumer',
  provider: 'MoviesAPI',
});

const EXPECTED_BODY = { id: 1, name: "My movie", year: 1999 };
describe('Movies Service', () => {
  describe('When a GET request is made to /movies', () => {
    test('it should return all movies', async () => {
      provider
        .uponReceiving('a request to all movies')
        .withRequest({
          method: 'GET',
          path: '/movies',
        })
        .willRespondWith({
          status: 200,
          body: eachLike(EXPECTED_BODY),
        });

      await provider.executeTest(async mockProvider => {
        const movies = await fetchMovies(mockProvider.url);
        expect(movies[0]).toEqual(EXPECTED_BODY);
      });
    });
  });

  describe('When a GET request is made to a specific movie ID', () => {
    test('it should return a specific movie', async () => {
      const testId = 100;
      EXPECTED_BODY.id = testId;

      provider
        .given('Has a movie with specific ID', { id: testId })
        .uponReceiving('a request to a specific movie')
        .withRequest({
          method: 'GET',
          path: `/movie/${testId}`,
        })
        .willRespondWith({
          status: 200,
          body: {
            id: integer(testId),
            name: string(EXPECTED_BODY.name),
            year: integer(EXPECTED_BODY.year)
          }
        });

      await provider.executeTest(async mockProvider => {
        const movies = await fetchSingleMovie(mockProvider.url, testId);
        expect(movies).toEqual(EXPECTED_BODY);
```